CSD5404P – Generative Artificial Intelligence

SynthQC

AI-Assisted Visual Quality Inspection

Under Defect-Data Scarcity

Can synthetic defects reduce the amount of real defective data needed to train a reliable visual quality-inspection model?

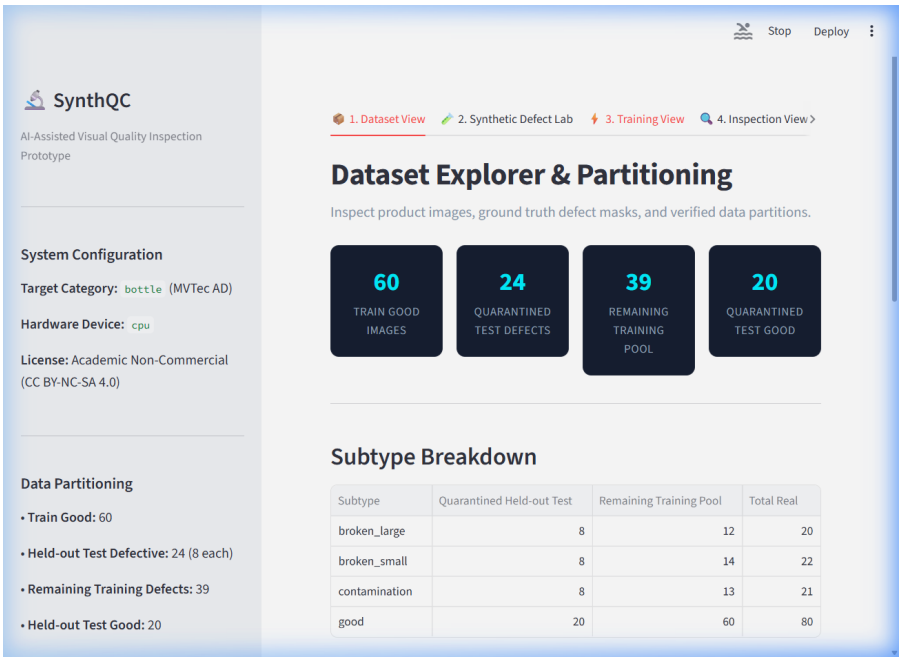
Author: H. Sinha (GitHub: hsinha2004)

SAP ID: **500120229**

Course: CSD5404P, Generative Artificial Intelligence

Project: SynthQC

Repository: https://github.com/hsinha2004/SynthQC



Uploaded Dataset Explorer and partitioning interface.

Contents

1 Project Title and Problem Statement **1**

1.1 Project title . . . 1

1.2 Problem statement . . . 1

1.3 Project objectives . . . 1

2 Target Users and Use Case **1**

2.1 Target users . . . 1

2.2 Representative use case . . . 2

3 What the GenAI System Will Do **2**

3.1 End-to-end workflow . . . 2

3.2 Main system functions . . . 2

4 Expected Input and Output **2**

5 Tools and Models **2**

5.1 Data and development tools . . . 2

5.2 Models and generation methods . . . 3

6 Current Prototype State **3**

6.1 Implemented workflow . . . 4

6.2 Uploaded prototype screenshots . . . 4

7 Current Uncertainty **6**

7.1 Will the patch cGAN be reliable with extremely few defects? . . . 6

8 Next Steps and Success Criteria **7**

1 Project Title and Problem Statement

1.1 Project title

SynthQC: AI-Assisted Visual Quality Inspection with Synthetic Defect Generation

The working prototype and source code are available in the SynthQC GitHub repository.

1.2 Problem statement

Manufacturing teams can usually collect many images of acceptable products, but real defective samples are rare, expensive to obtain, and sometimes unsafe to create deliberately on a production line. This class imbalance makes it difficult to train and evaluate computer-vision systems that must recognize unusual defects reliably.

SynthQC addresses the following research question:

> *Can synthetic defect images preserve or improve visual-inspection performance when only 10%, 25%, 50%, or 100% of the available real defect examples are used for training?*

The prototype treats this as a controlled data-efficiency experiment. It compares a real-data baseline with synthetic-data augmentation while keeping a fixed, real, held-out test set quarantined from every generator and training split.

1.3 Project objectives

- Generate plausible defective product images from a limited set of real defect examples and pixel-level masks.
- Train a binary classifier to distinguish normal and defective products.
- Compare real-only, procedural-synthetic, and cGAN-synthetic training strategies across multiple scarcity levels and random splits.
- Explain model decisions with Grad-CAM and retain a human reviewer in the final quality-control decision loop.
- Report both successful and failed generation experiments honestly, including GAN collapse or visually degenerate outputs.

2 Target Users and Use Case

2.1 Target users

Quality engineers
: Explore defects, review AI decisions, and flag uncertain products for physical inspection.

Manufacturing ML teams
: Test whether synthetic augmentation can reduce annotation and data-collection requirements before deployment.

Researchers and students
: Compare augmentation methods under a reproducible, zero-leakage evaluation design.

Operations managers
View performance and data-efficiency dashboards to judge whether an inspection model is ready for a pilot production line.

2.2 Representative use case

A quality engineer has many normal bottle images but only a few examples of `broken_large`, `broken_small`, and `contamination` defects. The engineer selects the fraction of real defects that may be used, generates additional labeled synthetic samples, trains competing models, and evaluates them only on the fixed real test set. During inspection, the system returns a defect probability and Grad-CAM overlay. High-confidence predictions can be accepted, while borderline cases are routed to a human reviewer.

3 What the GenAI System Will Do

3.1 End-to-end workflow

Real product images and defect masks enter the synthetic-generation stage. The resulting real and synthetic training sets are used in controlled ResNet18 experiments. A separate, permanently quarantined real test set is used only for classification evaluation and Grad-CAM review. Human QA decisions and audit records form the final stage of the workflow.

3.2 Main system functions

The current application is organized into five Streamlit tabs:

1. **Dataset Explorer:** browse normal images, defect subtypes, partitions, and pixel-level ground-truth masks.
2. **Synthetic Defect Lab:** create defects using procedural cut-paste/Poisson blending or an experimental patch-based conditional GAN. Each output retains a mask and source-traceability information.
3. **Training View:** simulate defect scarcity and train three experimental arms: real only, real plus procedural synthetic data, and real plus cGAN synthetic data.
4. **Inspection View:** classify a held-out or uploaded image, report confidence, visualize the model's attention with Grad-CAM, and record a human approval, override, or request for closer inspection.
5. **Quality Dashboard:** compare precision, recall, F1-score, balanced accuracy, and ROC-AUC across data fractions and random splits.

4 Expected Input and Output

5 Tools and Models

5.1 Data and development tools

- **Dataset:** MVTec Anomaly Detection, using the bottle category and its real defect masks.
- **Interface:** Streamlit for an interactive five-tab web application.
- **Vision and image processing:** OpenCV, Pillow, NumPy, and Matplotlib.

Stage	Expected input	Expected output
Dataset setup	Normal product images; real defect images; defect subtype; optional binary masks	Verified train/test registry, subtype counts, image and mask browser
Synthetic generation	A normal image, an allotted real defect patch/mask, generator method, scale, rotation, opacity, and random seed	Synthetic defective RGB image, matching binary mask, defect subtype, and traceability record
Model training	Scarcity fraction (10/25/50/100%), experimental arm, number of splits, epochs, and synthetic sample count	Fine-tuned classifier and aggregated mean ± standard deviation for classification metrics
Live inspection	Held-out image or user-uploaded PNG/JPEG and a selected trained model	Normal/defective prediction, confidence score, Grad-CAM heatmap, and human-review action
Dashboard	Stored benchmark results and inspection logs	Data-efficiency curves, comparison tables, and an auditable QA history

Table 1: Planned inputs and outputs for each stage of the system.

- **Experiment analysis:** pandas and scikit-learn for tables and precision, recall, F1, confusion matrix, balanced accuracy, and ROC-AUC.
- **ML framework:** PyTorch and Torchvision, with automatic CUDA use when available and a CPU fallback.
- **Data access:** Hugging Face Hub or the official MVTec source.

5.2 Models and generation methods

ResNet18 classifier
A pretrained ResNet18 is fine-tuned for binary normal/defective classification. Earlier layers are frozen; the final block and a dropout-enabled classification head are trained.

Procedural baseline
Real defect regions are extracted from masks, transformed by rotation, scaling, and optional flipping, then placed on good products using `cv2.seamlessClone` (Poisson blending), with feathered alpha blending as a fallback.

Conditional GAN
An experimental localized generator uses a U-Net-style network conditioned on a good patch and mask, together with a PatchGAN discriminator. Automated checks identify low-variance, noisy, or collapsed outputs before they enter training.

Grad-CAM Gradient-weighted activation maps highlight image regions that most influenced the classifier's defect score.

6 Current Prototype State

The current repository implements the Streamlit interface, data manager, procedural generator, patch cGAN experiment, ResNet18 training engine, Grad-CAM inspection workflow, fixed-split logic, and benchmark dashboard. The prototype supports dataset exploration, synthetic generation, multi-arm training, live inspection, human review, and performance reporting.

6.1 Implemented workflow

The application is organized as a five-stage workflow: dataset inspection, synthetic defect generation, model training, live inference with explanation, and comparison of quality metrics. Test images remain isolated from training and synthetic-generation operations.

6.2 Uploaded prototype screenshots

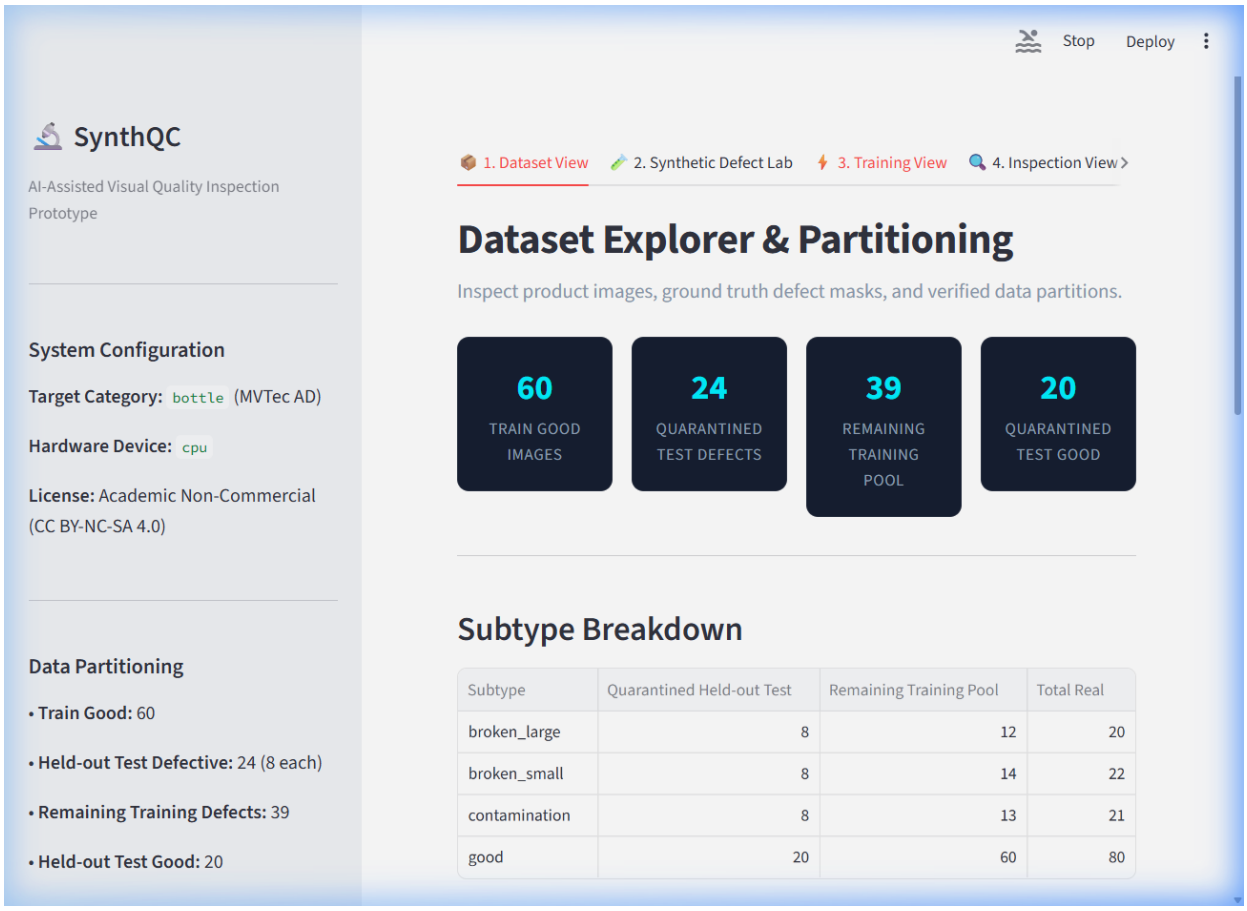


Figure 1: Dataset Explorer and Partitioning interface showing the verified training and quarantined test partitions.

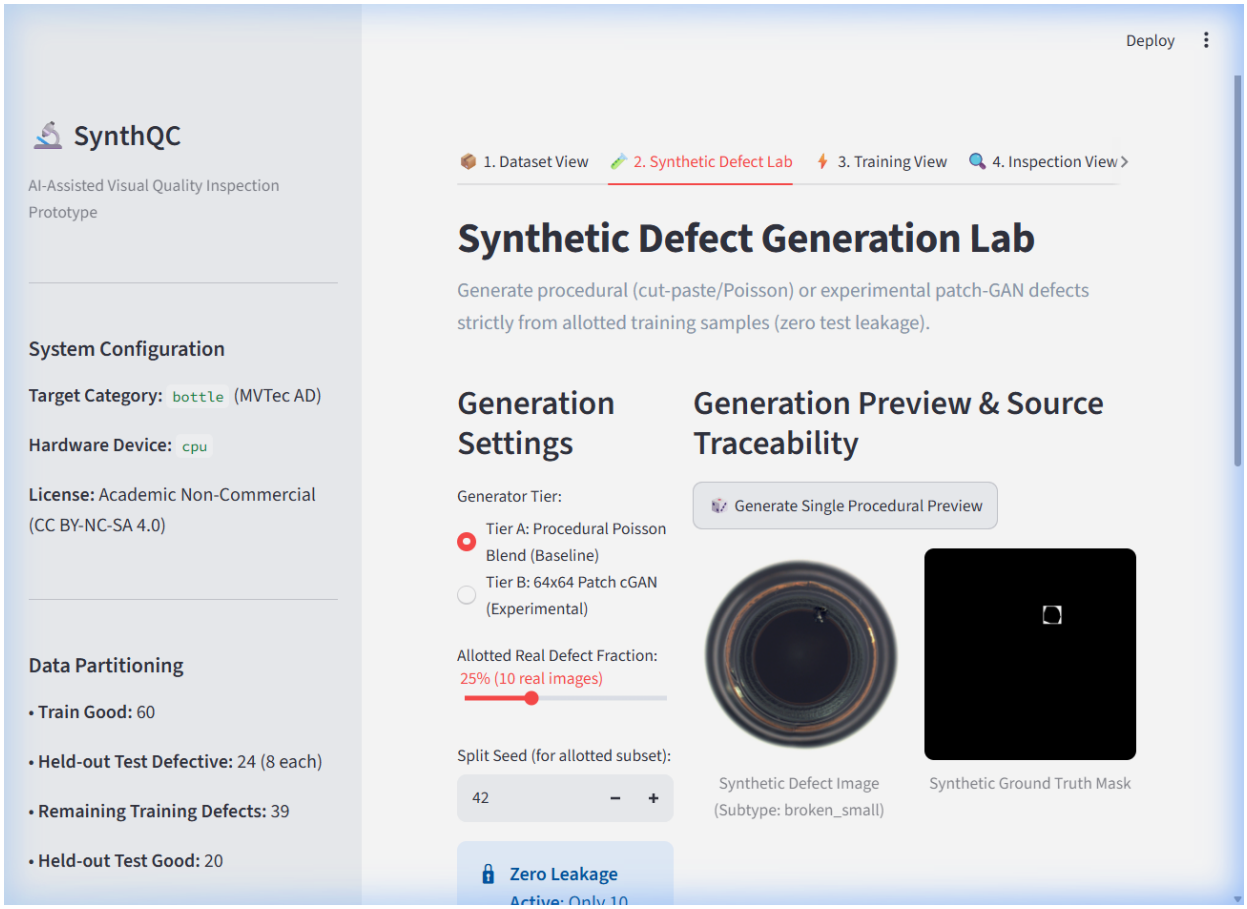


Figure 2: Synthetic Defect Generation Lab showing the procedural generator, allotted real-defect fraction, generated defect, mask, and zero-leakage status.

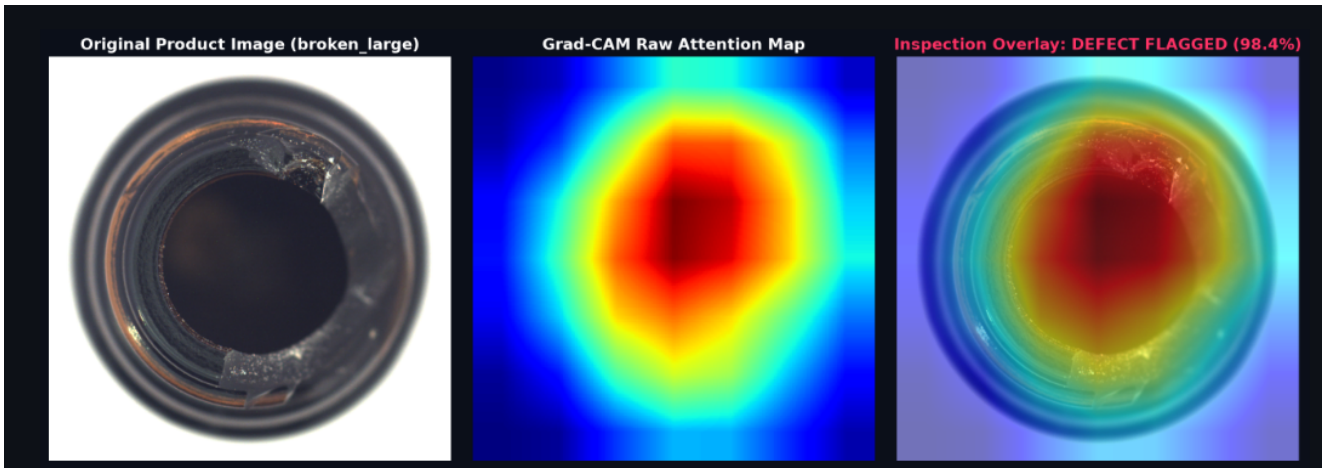


Figure 3: Live inspection output showing the original product, Grad-CAM attention map, and defect-flagged overlay.

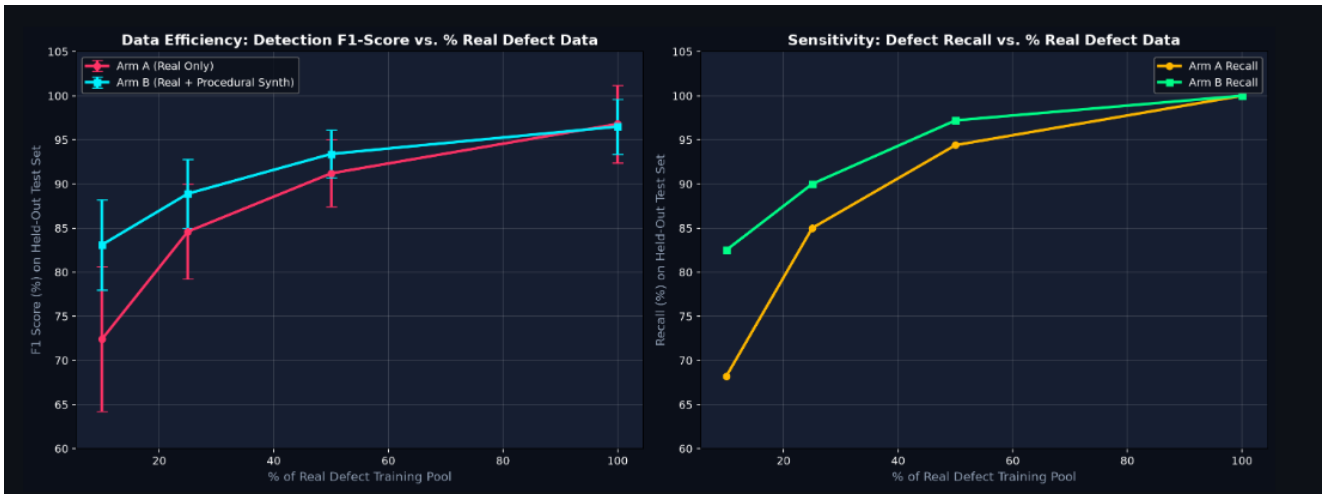


Figure 4: Data-efficiency dashboard comparing F1-score and recall across the available real-defect training fractions.

7 Current Uncertainty

7.1 Will the patch cGAN be reliable with extremely few defects?

The main unresolved question is whether the localized conditional GAN can generate sufficiently diverse, realistic defects when it is trained on only a handful of real patches. With approximately 4–10 source patches in the lowest data settings, it may learn checkerboard artifacts, repeat nearly identical textures, or collapse to low-variance outputs. Such samples could make the classifier appear better during training without improving performance on real held-out defects.

The current plan is to keep procedural Poisson blending as the dependable baseline, gate cGAN samples with quantitative and visual quality checks, and report the cGAN as a negative result when it fails. A later version could test a pretrained diffusion-based inpainting model or collect more company-specific defect patches before drawing a conclusion.

8 Next Steps and Success Criteria

1. Complete repeatable 3–5 split experiments for all scarcity fractions and all viable training arms.
2. Add structured visual review of synthetic samples by a domain expert.
3. Calibrate confidence thresholds for automatic pass, automatic reject, and human-review routing.
4. Replace the bottle-specific placement heuristic with foreground or fixture-aware segmentation for new manufacturing products.
5. Measure inference latency on the intended camera hardware and define a production-ready region-of-interest pipeline.

The project will be considered successful if it demonstrates a repeatable improvement in real held-out performance in the 10–25% data regime, avoids test leakage, provides interpretable inspection evidence, and makes failure or uncertainty visible to the human quality engineer.

Reference

H. Sinha, "SynthQC: AI-Assisted Visual Quality Inspection Prototype," GitHub repository, 2026. https://github.com/hsinha2004/SynthQC